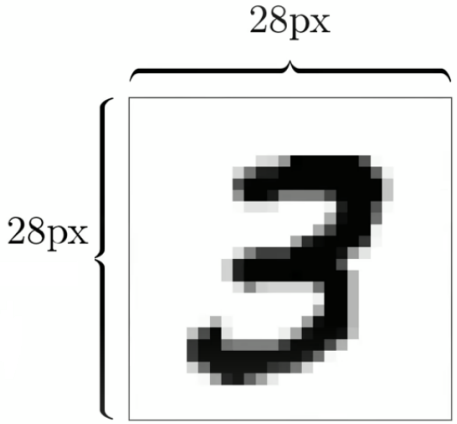
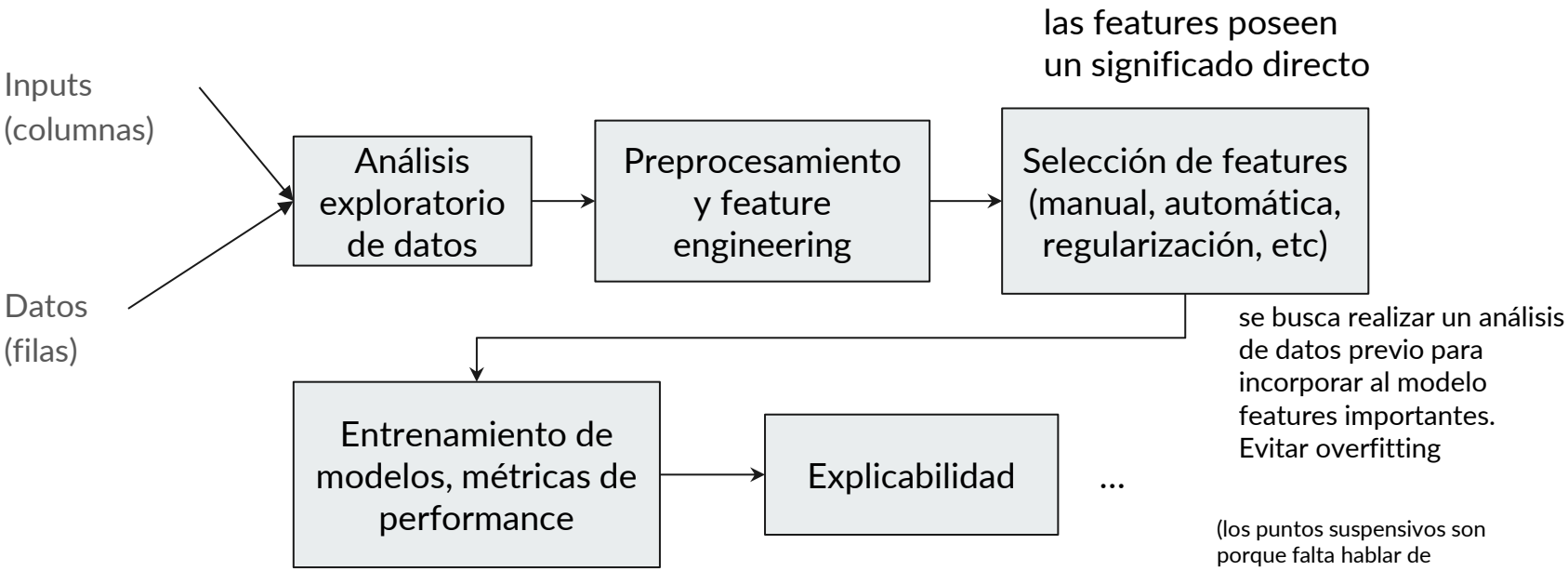
Introducción a Aprendizaje Profundo

Deep Learning




28px
28px

Solución de Aprendizaje de Máquina



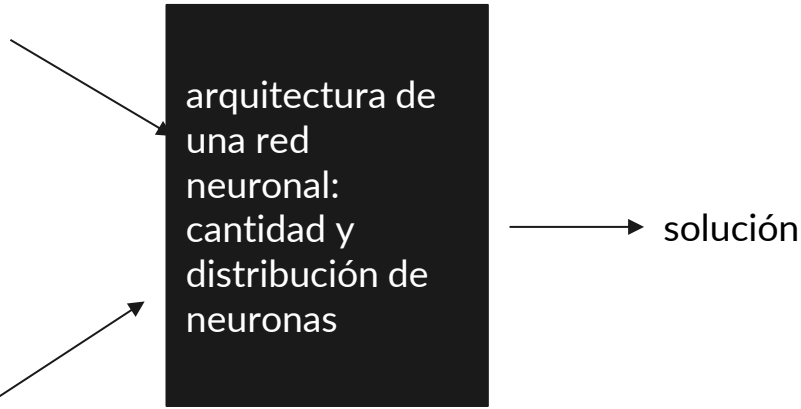
Solución de Aprendizaje Profundo

Inputs = 28 x 28 píxeles= 784 features

cada píxel de una imagen o cada palabra de un texto

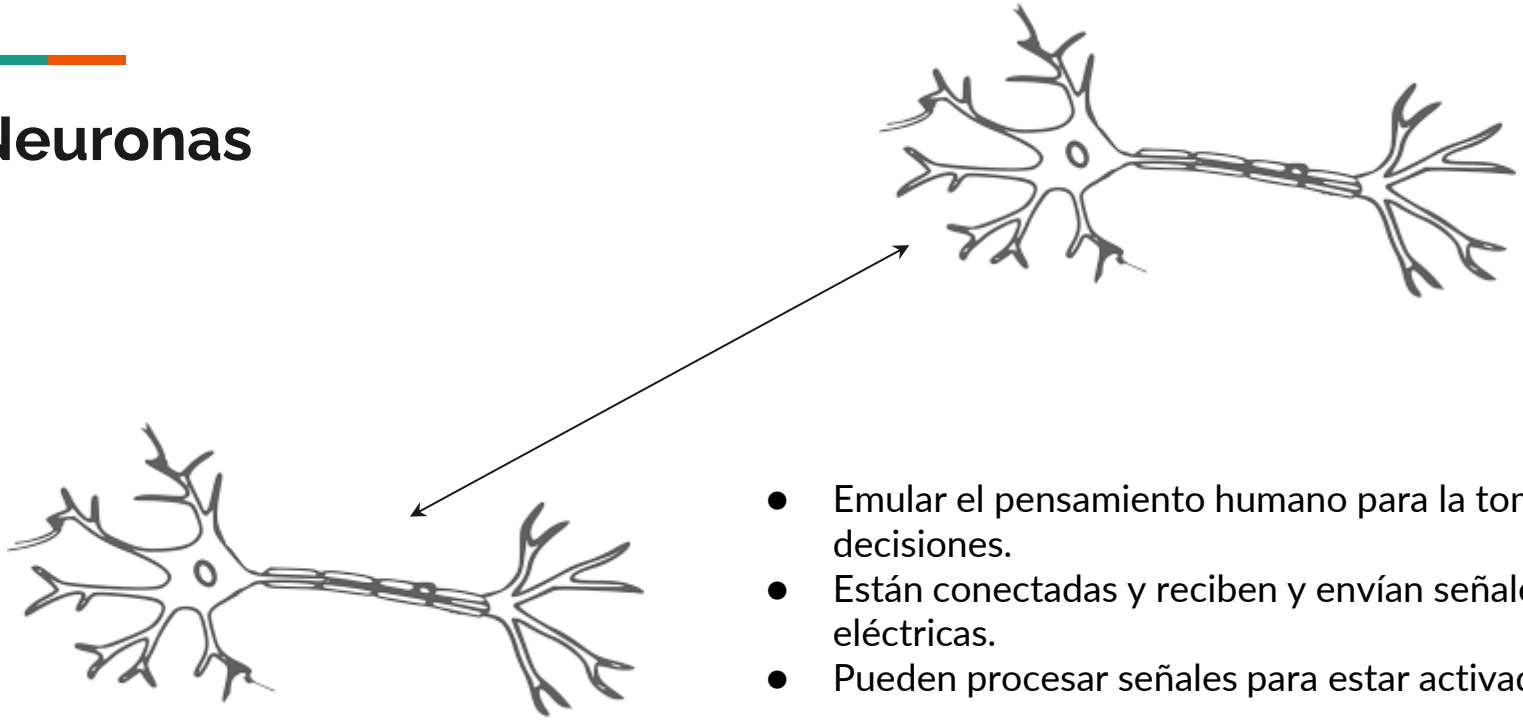
aprendizaje composicional

Datos = conjunto de entrenamiento y validación (hiperparámetros)



solución end-to-end

Neuronas



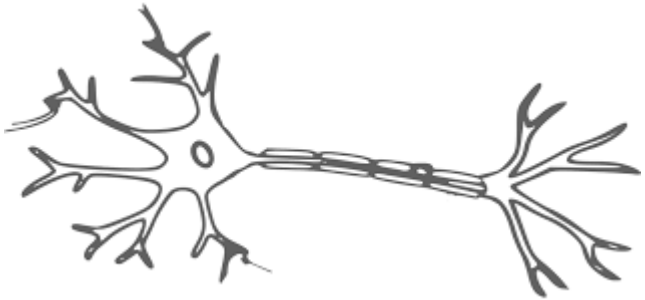
- Emular el pensamiento humano para la toma de decisiones.
- Están conectadas y reciben y envían señales eléctricas.
- Pueden procesar señales para estar activadas o no.

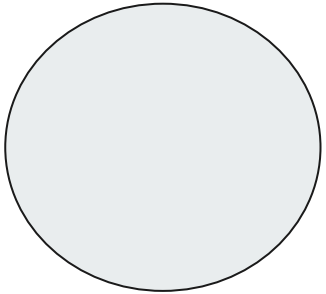
Neuronas

El objetivo es crear un modelo matemático que modele de las entradas hasta las salidas basado en una estructura de una red neuronal.

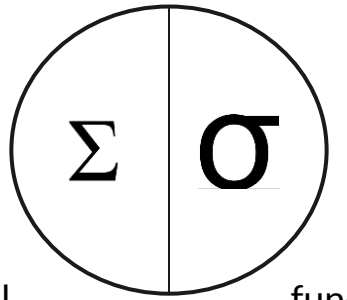
Esta red, modelada por funciones matemáticas, tendrá ciertos parámetros que deben aprenderse de la inmensidad de datos del conjunto de entrenamiento.

Abstracción de la neurona





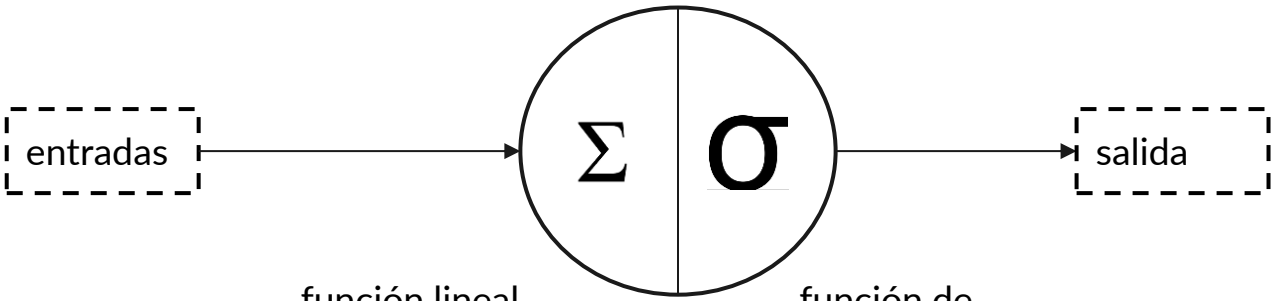
Abstracción de la neurona



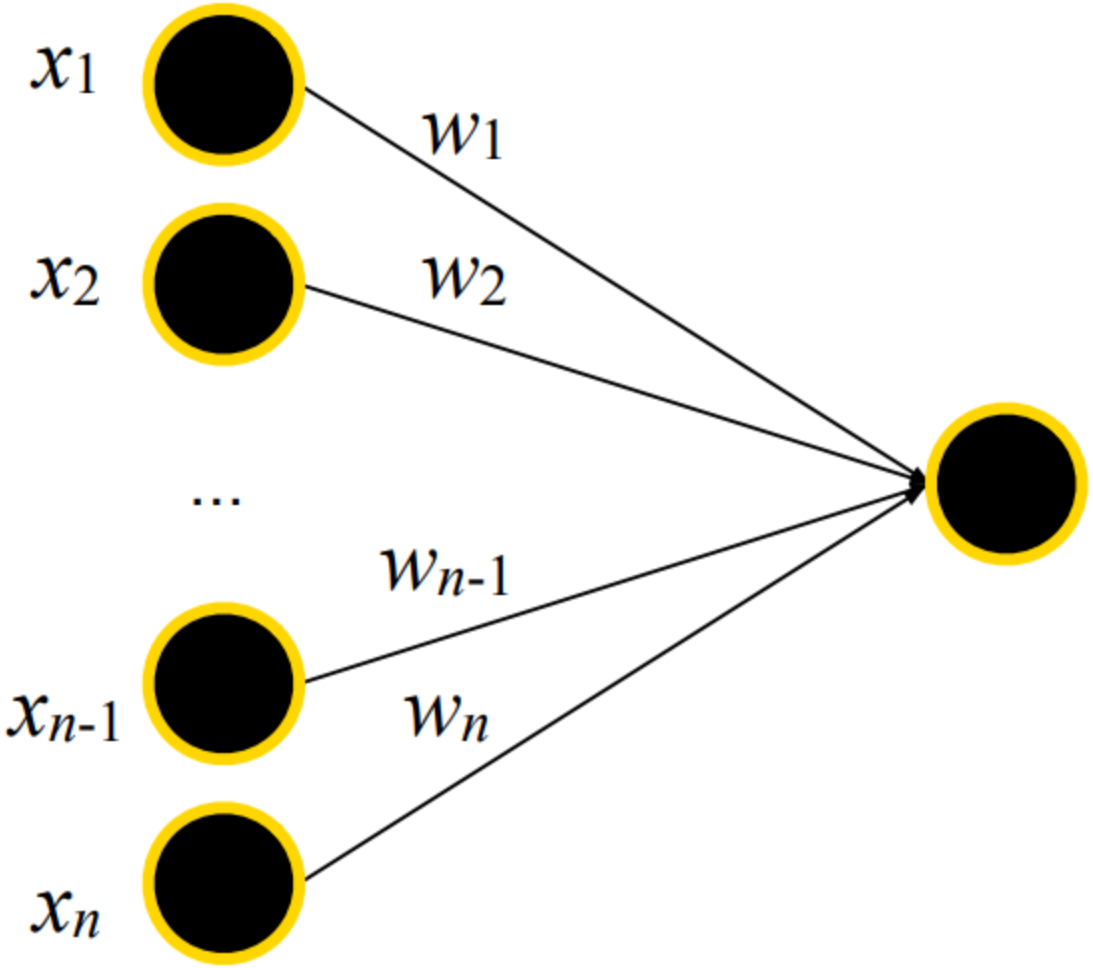
función lineal

función de activación

Abstracción de la neurona

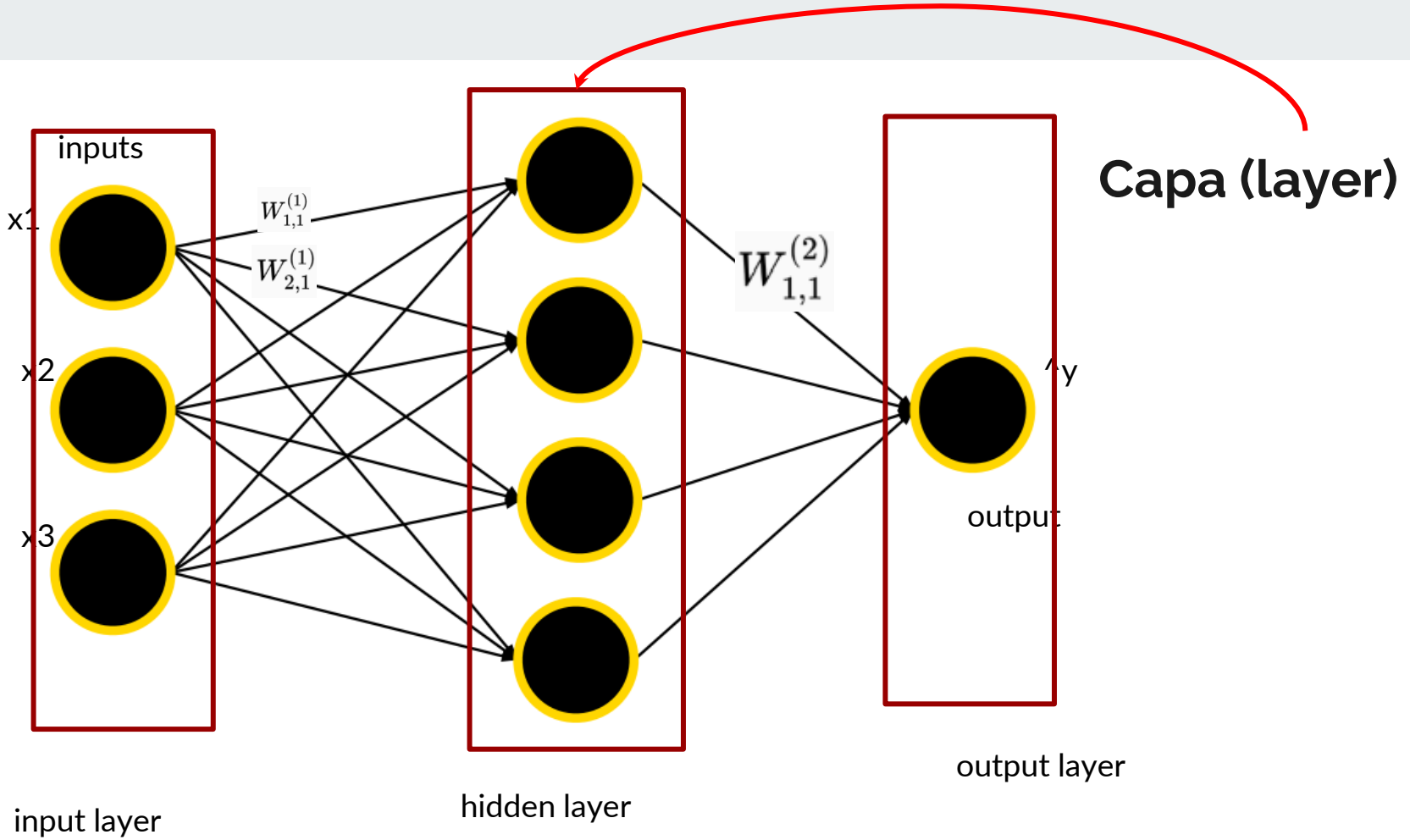


sigma(w0 + w1 x1 + w2 x2 + … wn xn)



$$g\left(\sum_{i=1}^{n} x_i w_i + w_0\right)$$

ecuaciones feed-forward


Capa (layer)
inputs
x1
x2
x3
$W^{(1)}_{1,1}$
$W^{(1)}_{2,1}$
$W^{(2)}_{1,1}$
^y
output
input layer
hidden layer
output layer

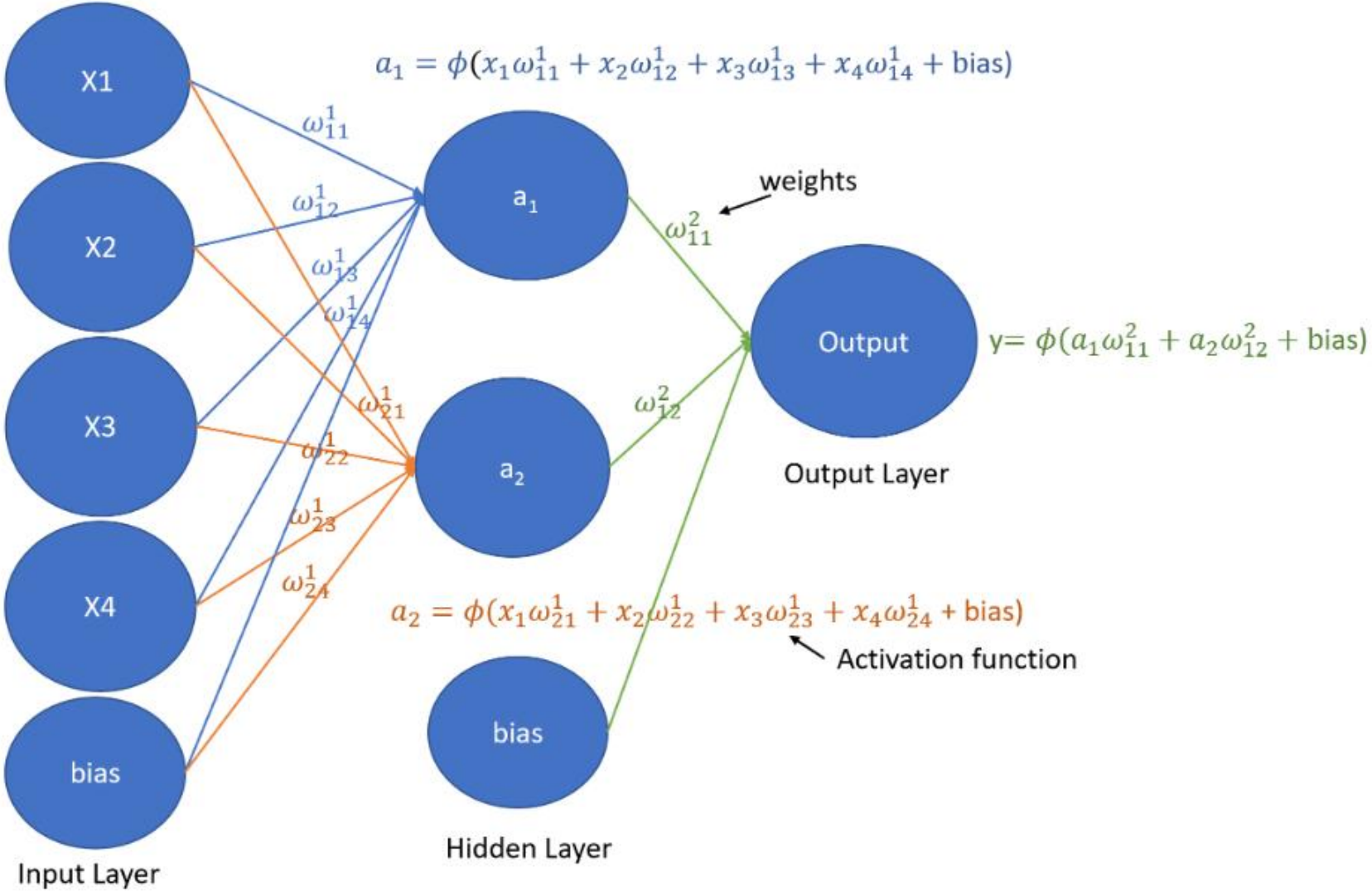
Pesos y biases de una red neuronal

Cada neurona se relaciona con las neuronas de la capa anterior a través de un peso.

A su vez cada neurona tiene un bias asociado, que sería el término independiente.

Los pesos se denominan W_{ij}^{k}

i: n° de neurona de la capa anterior
j: n° de neurona de la capa siguiente
k: n° de capa


$a_1 = \phi(x_1\omega_{11}^1 + x_2\omega_{12}^1 + x_3\omega_{13}^1 + x_4\omega_{14}^1 + \text{bias})$
X1
X2
X3
X4
bias
ω_{11}^1
ω_{12}^1
ω_{13}^1
ω_{14}^1
ω_{21}^1
ω_{22}^1
ω_{23}^1
ω_{24}^1
a_1
a_2
bias
weights
ω_{11}^2
ω_{12}^2
Output
$y = \phi(a_1\omega_{11}^2 + a_2\omega_{12}^2 + \text{bias})$
Output Layer
$a_2 = \phi(x_1\omega_{21}^1 + x_2\omega_{22}^1 + x_3\omega_{23}^1 + x_4\omega_{24}^1 + \text{bias})$
Activation function
Input Layer
Hidden Layer

Cantidad de parámetros de una red neuronal

Parámetros

En una regresión logística con 784 features, ¿cuantos **parámetros** había?

Parámetros

En una red neuronal con 4 variables explicativas, una capa oculta de 2 neuronas, y una sola output, ¿cuántos parámetros hay?

Tenemos 4 pesos en cada neurona de la capa oculta (uno por cada input).

Hay 2 biases en la capa oculta (uno por cada neurona).

Tenemos 2 pesos en la salida (uno por cada neurona de la capa anterior).

A su vez hay 1 bias.

En total (4*2+2) + (2*1 + 1) = 13 parámetros.

esto sería equivalente a una muy simple red neuronal que toma como entradas el valor de cada pixel (28x28 entradas) y como salida están los diez dígitos.

Parámetros

En una red neuronal con 784 variables explicativas, y una capa de salida de 10 outputs, ¿cuantos parámetros hay?

Tenemos 784 pesos en cada neurona de la capa de salida (784 pesos que provienen de las inputs y un bias) -> 784 * 10 = **7840 pesos.**

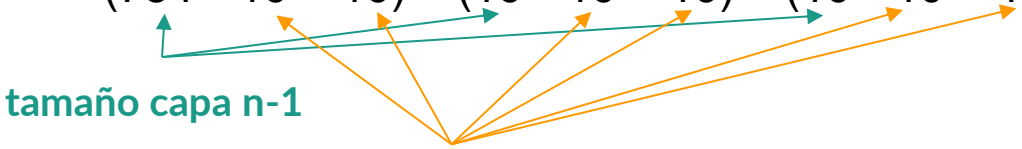
A su vez cada neurona en la capa de salida tiene un bias -> **10 biases.**

En total 7840 + 10 = **7850 parámetros.**

Parámetros

En una red neuronal con 784 variables explicativas, dos capas ocultas de 16 neuronas cada una, y una salida de 10 outputs, ¿cuántos parámetros hay?

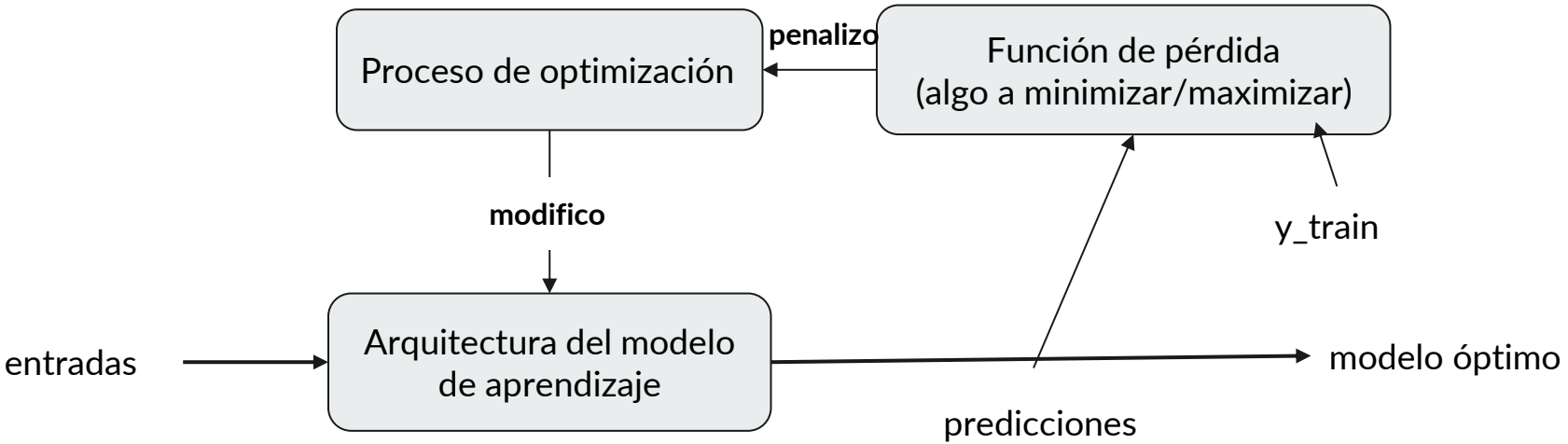
Siguiendo la lógica anterior, serán:
(784 * 16 + 16) + (16 * 16 + 16) + (16 * 10 + 10) = 13002 parámetros



para n= 1, 2 y 3 en este caso (dos capas ocultas, una de salida)

Entrenamiento de una red neuronal

Proceso de entrenamiento (ML/DL)



Proceso de entrenamiento: Función de pérdida

- Error cuadrático Medio
- Error porcentual medio
- Huber Loss
- Entropía binaria cruzada
- Entropía binaria

https://www.tensorflow.org/api_docs/python/tf/keras/losses

Proceso de entrenamiento: Arquitecturas

Tipos de layers

- Fully connected
- Convolucional
- Recurrente
- LSTM

Hiperparámetros

- n° de neuronas
- n° de capas

Funciones de activación

- ninguna
- sigmoide
- softmax
- tanh
- ReLu
- Leaky ReLu

https://www.tensorflow.org/api_docs/python/tf/keras/layers

Proceso de entrenamiento: optimización

Algoritmos de optimización

- **Gradiente descendiente** (con o sin momento). Por lo general **mini-batch**
- AdaGrad
- RMSProp
- Adam

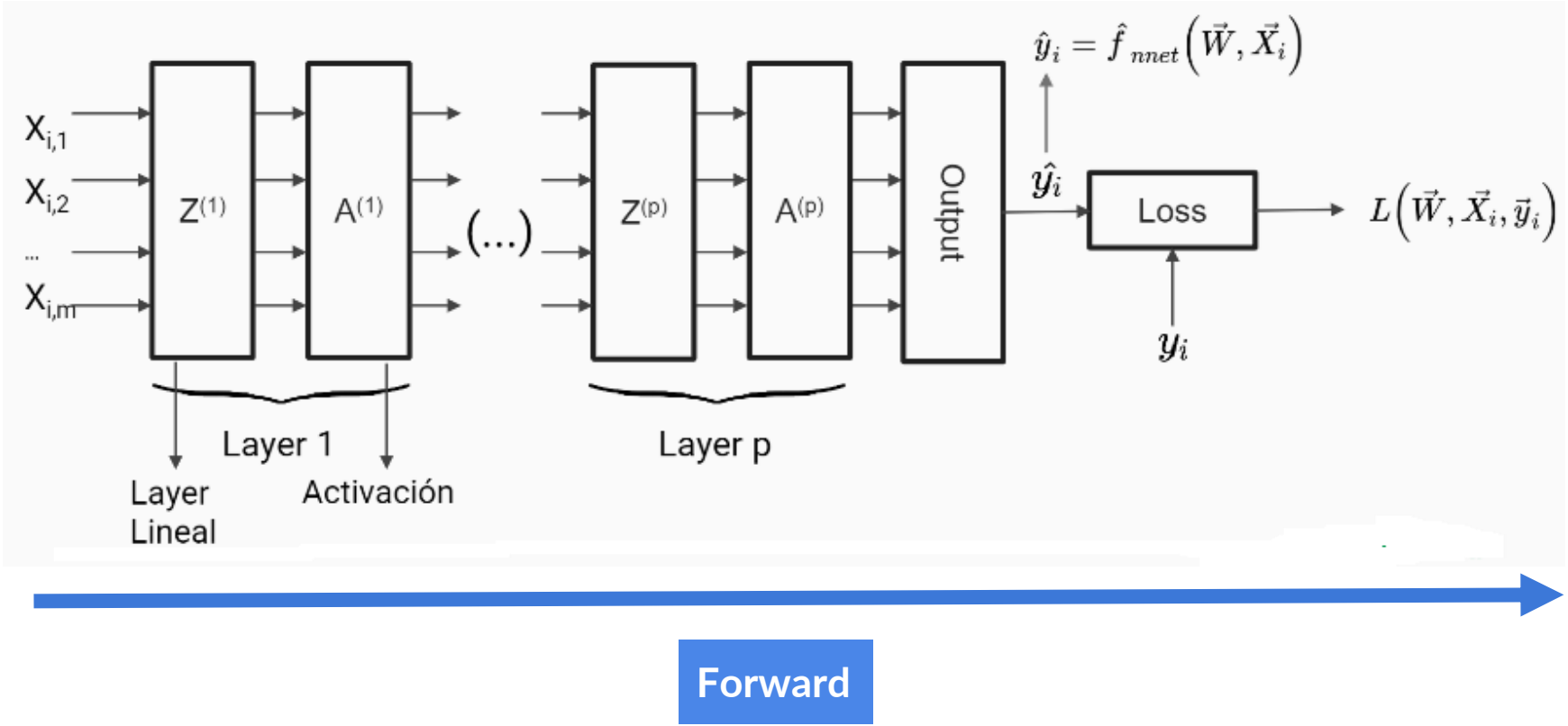
https://www.tensorflow.org/api_docs/python/tf/keras/optimizers

Ecuaciones feed-forward

así se denomina a todo el conjunto de ecuaciones

$$a_1 = \phi(x_1\omega_{11}^1 + x_2\omega_{12}^1 + x_3\omega_{13}^1 + x_4\omega_{14}^1 + \text{bias})$$

Eventualmente se llega a una ecuación de salida que claramente depende de todos los parámetros de la red.


$\hat{y}_i = \hat{f}_{nnet}(\vec{W}, \vec{X}_i)$
$X_{i,1}$
$X_{i,2}$
...
$X_{i,m}$
$Z^{(1)}$
$A^{(1)}$
(...)
$Z^{(p)}$
$A^{(p)}$
Output
$\hat{y}_i$
Loss
$L(\vec{W}, \vec{X}_i, \vec{y}_i)$
y_i
Layer 1
Layer p
Layer Lineal
Activación
Forward

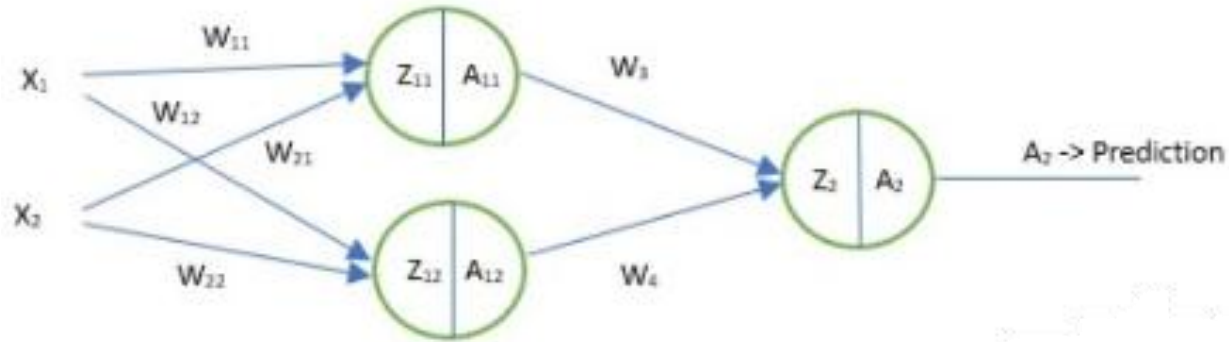
Ok y… cómo optimizo el proceso con Gradiente descendiente u otro método?

- Para cada parámetro (de los que pueden ser miles o millones!) se deriva la función de pérdida respecto del parámetro.
- Se computa con ello el gradiente (que es un vector de tamaño n°parámetros).
- Nos movemos en la **dirección opuesta al gradiente.**
- Actualizamos los parámetros acorde a un **learning_rate**.

estos tres pasos ya los hemos hecho

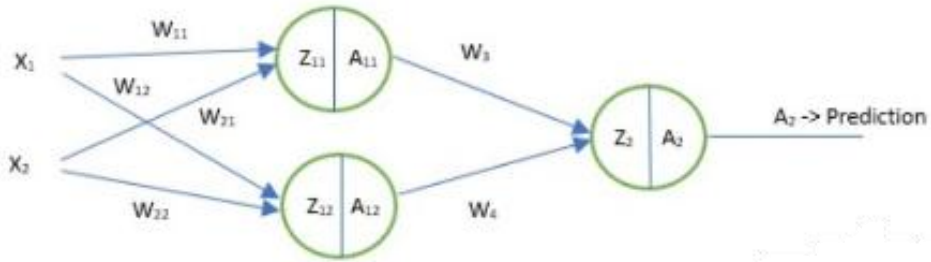
nos faltaría ver cuánto dan las derivadas

Ejemplo



9 parámetros

Ecuaciones forward



$$Z_{i,1}^{(1)} = W_{1,1}^{(1)} \cdot X_{i,1} + W_{1,2}^{(1)} \cdot X_{i,2} + b_1^{(1)}$$

$$Z_{i,2}^{(1)} = W_{2,1}^{(1)} \cdot X_{i,1} + W_{2,2}^{(1)} \cdot X_{i,2} + b_2^{(1)}$$

$$a_{i,1}^{(1)} = \sigma\left(Z_{i,1}^{(1)}\right)$$

$$a_{i,2}^{(1)} = \sigma\left(Z_{i,2}^{(1)}\right)$$

$$Z_{i,1}^{(2)} = W_{1,1}^{(2)} \cdot a_{i,1}^{(1)} + W_{1,2}^{(2)} \cdot a_{i,2}^{(1)} + b_1^{(2)}$$

$$\hat{y}_i = a_{i,\mathbf{1}}^{(2)} = Z_{i,1}^{(2)}$$

$$L_{\vec{W}} = (y_i - \hat{y}_i)^2$$

Optimización de pesos y biases

- Inicializo los pesos y biases en un valor aleatorio.
- Para cada iteración:
  - Para cada fila del dataset de entrenamiento (SGD) o conjunto de filas (Mini-batch GD):
    - Obtengo, con el paso forward, la predicción.
    - Comparo esa predicción con el valor real
    - Obtengo un error dado por la función de pérdida. En este caso el MSE.
    - Calculo el gradiente

$$\partial L/\partial W_{1,1}^{(1)},\ \partial L/\partial W_{1,2}^{(1)}\ \partial L/\partial b_1^{(1)}$$
$$\partial L/\partial W_{2,1}^{(1)},\ \partial L/\partial W_{2,2}^{(1)},\ \partial L/\partial b_2^{(1)}$$
$$\partial L/\partial W_{1,1}^{(2)},\ \partial L/\partial W_{1,2}^{(2)},\ \partial L/\partial b_1^{(2)}$$

Optimización de pesos y biases

- Actualizo los pesos y biases

$$W_{1,1}^{(1)} \leftarrow W_{1,1}^{(1)} - \alpha \cdot \partial L / \partial W_{1,1}^{(1)}$$

$$\dots$$

$$b_1^{(2)} \leftarrow b_1^{(2)} - \alpha \cdot \partial L / \partial b_1^{(2)}$$

$$\sigma(x) = \frac{1}{1+e^{-x}} = \frac{e^x}{e^x+1}$$

$$\sigma'(x) = \sigma(x) \cdot (1-\sigma(x))$$

Ecuaciones Backpropagation

$$\partial L/\partial W_{1,1}^{(2)} = \frac{\partial}{\partial W_{1,1}^{(2)}}\left((y_i - \hat{y}_i)^2\right) = \partial L/\partial \hat{y}_i \cdot \partial \hat{y}_i/\partial W_{1,1}^{(2)} = 2 \cdot (y_i - \hat{y}_i) \cdot -1 \cdot a_{i,1}^{(1)}$$

$$\vdots$$

$$\partial L/\partial W_{1,1}^{(1)} = \partial L/\partial \hat{y}_i \cdot \partial \hat{y}_i/\partial a_{i,1}^{(1)} \cdot \partial a_{i,1}^{(1)}/\partial Z_{i,1}^{(1)} \cdot \partial Z_{i,1}^{(1)}/\partial W_{1,1}^{(1)} =$$

$$= -2 \cdot (y_i - \hat{y}_i) \cdot W_{1,1}^{(2)} \cdot \sigma\left(Z_{i,1}^{(1)}\right) \cdot \left(1 - \sigma\left(Z_{i,1}^{(1)}\right)\right) \cdot X_{i,1}$$

voy haciendo múltiples reglas de la cadena!

Ecuaciones backpropagation

Hay una ecuación de backpropagation por cada parámetro.

Es importante que los cálculos sean eficientes para obtener soluciones que

https://playground.tensorflow.org